Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский технологический университет
«МИСИС»

Институт компьютерных наук (ИКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

Лабораторная работа №2

по дисциплине «Технологии программирования»

на тему «Основы объектно-ориентированного программирования»

Выполнила:
студент группы БИВТ-25-4-5
Сафарян Элен Врежовна

Цель работы: изучить основные принципы объектно-ориентированного программирования — инкапсуляцию, наследование, полиморфизм и абстракцию; получить практические навыки проектирования иерархии классов на языке C# (.NET) с использованием абстрактных классов и интерфейсов на примере разработки базовых классов пошаговой стратегии.

1. Ход выполнения работы:

1.1. Проектирование иерархии классов:

По условию задания была спроектирована следующая иерархия на языке C#. Базовым классом для всех игровых объектов является абстрактный класс GameObject, хранящий свойства Id, Name, X и Y. От него наследуются два абстрактных класса: Unit (юнит со свойствами Alive и Hp) и Building (постройка со свойством Built). Классы GameObject, Unit и Building объявлены абстрактными (abstract class), поскольку в игре создаются только объекты конкретных типов, а не их абстрактные сущности. Поведение, общее для нескольких веток иерархии — способность атаковать и способность перемещаться — вынесено в интерфейсы IAttacker и IMoveable.

Конкретные классы: Archer (Unit + Attacker + Moveable) — стреляет из лука и перемещается по карте; Fort (Building + Attacker) — постройка, стреляющая из пушек, но не способная двигаться; MobileHouse (Building + Moveable) — постройка, способная передвигаться, но не атакующая. Такое разделение демонстрирует, что Attacker и Moveable — это именно независимые интерфейсы, а не часть основной иерархии наследования.

1.2. Реализация базового абстрактного класса GameObject и интерфейсов IAttacker, IMoveable:

Класс GameObject содержит защищённые свойства Id, Name, X, Y с автоматическими геттерами и сеттерами, а также публичные методы-геттеры getId(), getName(), getX(), getY(). Интерфейсы IAttacker и IMoveable объявляют контракты для методов attack(Unit unit) и move(). Ниже представлен код базовых структур проекта:

```
public abstract class GameObject
{
    2 usages
    protected int Id { get;  set; }
    2 usages
    protected string Name { get;  set;}
    2 usages
    protected int X { get; set;}
    2 usages
    protected int Y { get; set;}

    public int getId() => Id;
    public string getName() => Name;
    public int getX() => X;
    public int getY() => Y;

    2 usages
    public GameObject(int id, string name, int x, int y)
    {
        Id = id;
        Name = name;
        X = x;
        Y = y;
    }
}
```

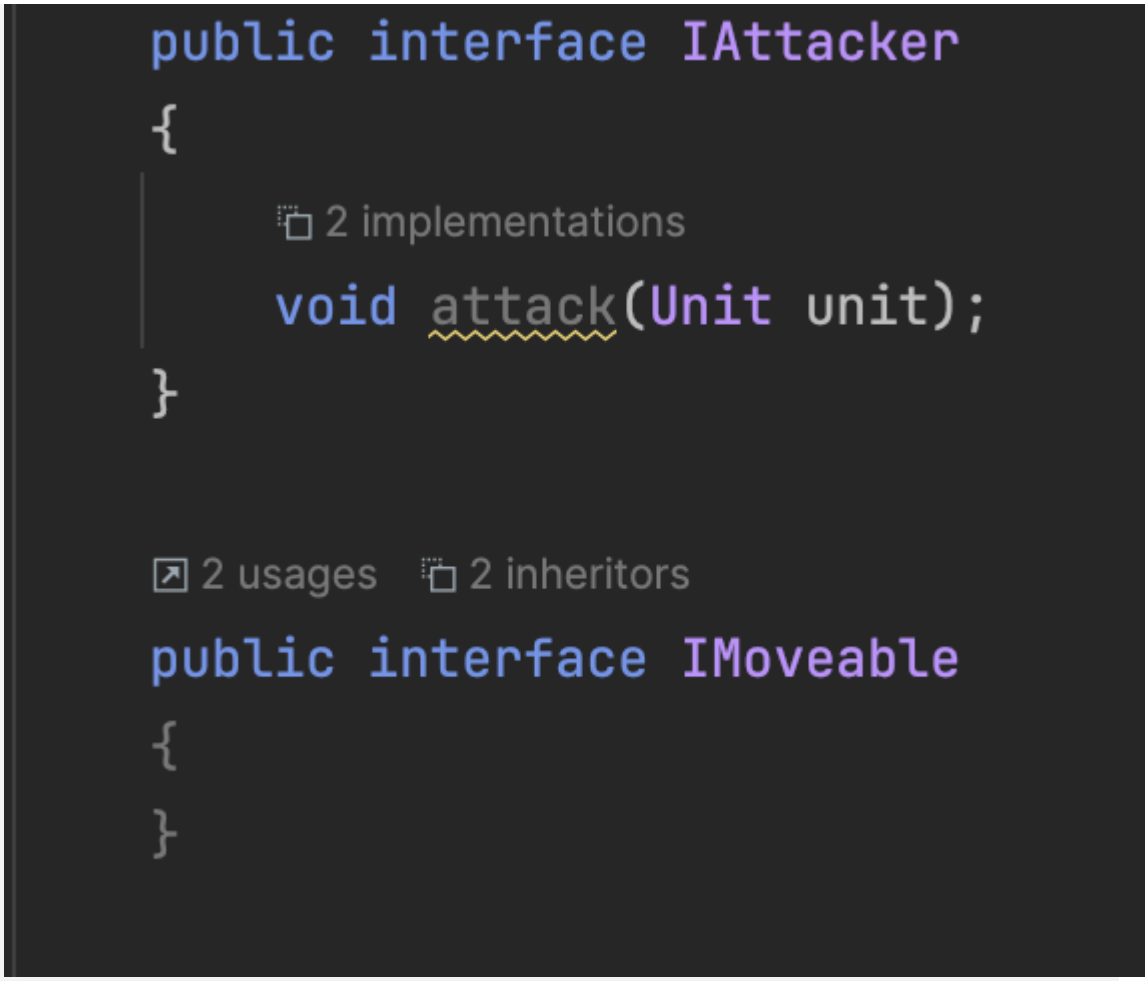
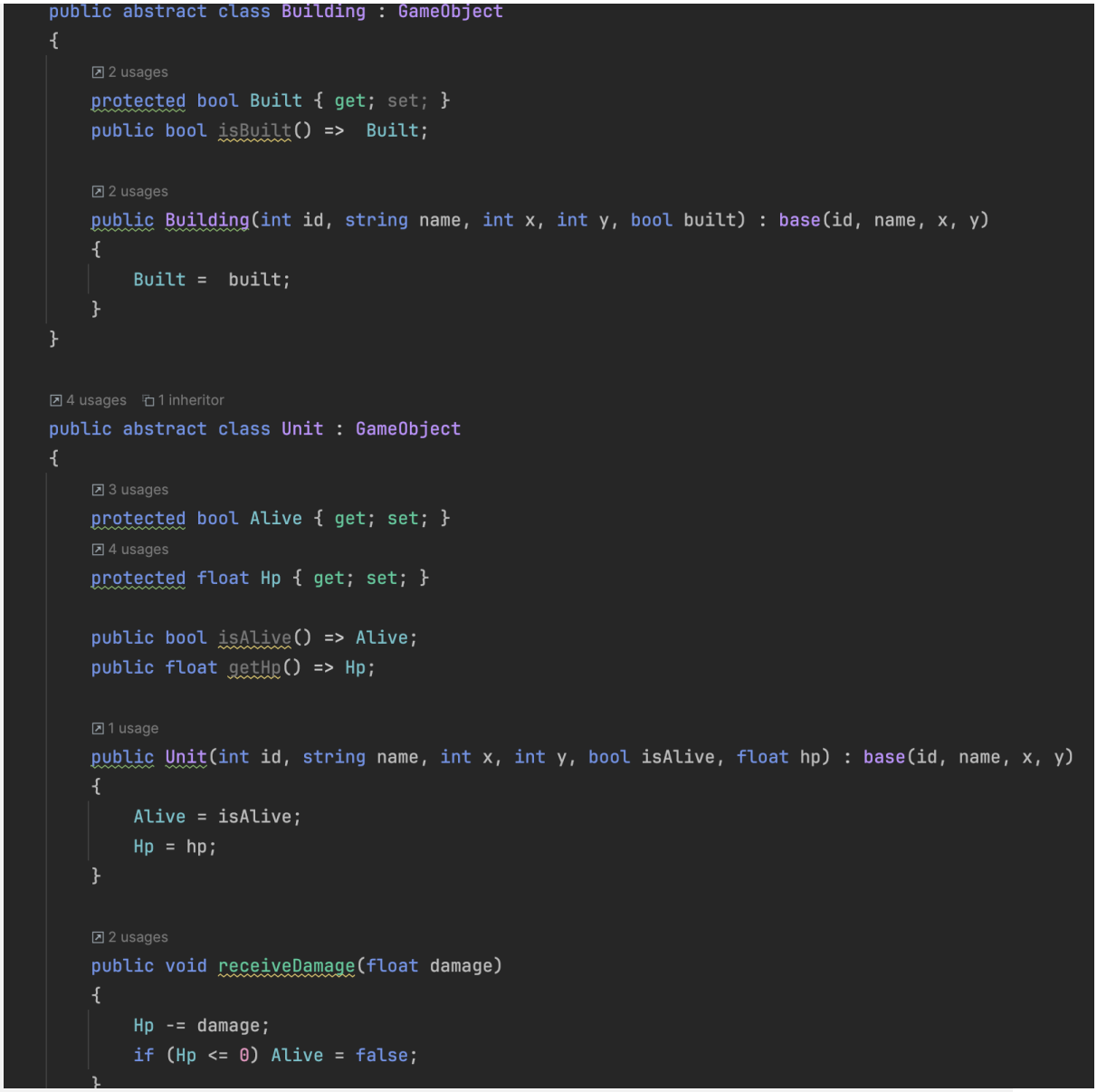


Рисунок 1 – Реализация GameObject.cs, IAttacker.cs и IMoveable.cs

1.3. Реализация абстрактных классов Unit, Building и их наследников:

Класс Building объявляет свойства Built и метод isBuilt(). Класс Unit объявляет свойства Alive, Hp и методы isAlive(), getHp(), receiveDamage(float damage). Метод receiveDamage списывает урон и при достижении Hp <= 0 устанавливает свойство Alive = false. Ниже приведена реализация классов Unit и Building:



```
public abstract class Building : GameObject
{
    2 usages
    protected bool Built { get; set; }
    public bool isBuilt() =>  Built;

    2 usages
    public Building(int id, string name, int x, int y, bool built) : base(id, name, x, y)
    {
        Built =  built;
    }
}

4 usages  1 inheritor
public abstract class Unit : GameObject
{
    3 usages
    protected bool Alive { get; set; }
    4 usages
    protected float Hp { get; set; }

    public bool isAlive() => Alive;
    public float getHp() => Hp;

    1 usage
    public Unit(int id, string name, int x, int y, bool isAlive, float hp) : base(id, name, x, y)
    {
        Alive = isAlive;
        Hp = hp;
    }

    2 usages
    public void receiveDamage(float damage)
    {
        Hp -= damage;
        if (Hp <= 0) Alive = false;
    }
```

Рисунок 2 – Реализация абстрактных классов Building.cs и Unit.cs

1.4. Реализация конкретных классов Fort, MobileHome и Archer:

Класс Fort наследует Building и реализует интерфейс IAttacker (вызывая receiveDamage(15) у цели). Класс MobileHome наследует Building и реализует интерфейс IMoveable. Класс Archer наследует Unit и реализует оба интерфейса IAttacker и IMoveable (вызывая receiveDamage(7) у цели). Ниже приведена их реализация:

```
public class Fort : Building, IAttacker
{
    public Fort(int id, string name, int x, int y, bool built) : base(id, name, x, y, built)
    {
    }

    public void attack(Unit unit)
    {
        unit.receiveDamage(15);
    }
}

public class MobileHome : Building, IMoveable
{

    public MobileHome(int id, string name, int x, int y, bool built) : base(id, name, x, y, built)
    {
    }
}

public class Archer : Unit, IAttacker, IMoveable
{

    public Archer (int id, string name,  int x, int y, bool isAlive, float hp) : base(id, name, x, y, isAlive, hp){}

    public void attack(Unit unit)
    {
        unit.receiveDamage(7);
    }
}
```

Рисунок 3 – Реализация классов Fort, MobileHome и Archer

1.5. Публикация решения в удалённом репозитории:

Для публикации выполненной работы в удалённом репозитории GitHub (https://github.com/iravasu/lab1proga) был выполнен последовательный процесс инициализации локального репозитория, индексации файлов, фиксации изменений и отправки кода. Ниже приведены основные этапы и соответствующие команды терминала:

1. Инициализация локального репозитория:

```
git init
```

2. Индексация файлов:

```
git add .
```

3. Фиксация изменений:

```
git commit -m "lab2"
```

4. Переименование ветки в main:

```
git branch -M main
```

5. Привязка удалённого репозитория и отправка изменений:

```
git remote add origin git@github.com:iravasu/lab1proga.git
git push -u origin main
```